LAB-03 REPORT

EMBEDDED LINUX SYSTEMS

Student name	Trương Ngọc Anh Khoa
Student ID	SE205096
Lab	LAB-03: Debugging, Memory Analysis and Performance Profiling
Environment	Ubuntu Linux in VirtualBox
Project directory	~/embedded_lab3

Prepared as a summary of the LAB-03 activities and the result files collected in the project folder.

1. Introduction

The purpose of this lab is to practice common Linux application debugging and performance-analysis tools. The work focuses on observing program execution, identifying memory-management problems, inspecting system calls, and collecting performance statistics. The project files are organized under ~/embedded_lab3, with application source code in apps/, text logs in logs/, generated outputs in output/, and this report in bao_cao/.

2. Objectives

- Compile C programs with debugging information so that a debugger can inspect source lines and variables.
- Use GDB to follow program execution and investigate incorrect behavior.
- Use Valgrind to detect invalid memory operations and memory leaks, then compare a leaky program with a corrected version.
- Use strace to observe system calls made by a Linux process.
- Use perf-related reports and histogram output to review program performance.

3. Environment and Project Structure

Item	Description
Operating system	Ubuntu Linux running in VirtualBox
Language	C
Main tools	GDB, Valgrind, strace, perf and GCC
Source files	buggy_app.c, leaky_app.c, leaky_app_fixed.c, perf_target.c
Evidence folder	logs/ contains text outputs for the analysis steps

4. Debugging with GDB

GDB is used to run a program under debugger control. With a build that includes debugging symbols (for example, compiled with -g -O0), the user can set breakpoints, step through statements, inspect variables, and obtain a backtrace when execution stops.

Typical workflow:

- Compile the source with debugging symbols: gcc -g -O0 -Wall -Wextra -o buggy_app buggy_app.c
- Start GDB: gdb ./buggy_app
- Set a breakpoint at main, run the program, and use commands such as break main, run, next, print variable, and backtrace.

Evidence: logs/gdb_session.txt. The file should be used as the exact record of commands and observations made during the debugging session.

5. Memory Analysis with Valgrind

Valgrind Memcheck checks heap-memory usage, including leaked allocations, invalid reads and writes, and use of uninitialized values. The baseline application leaky_app.c is intended to demonstrate allocations that are not released. The corrected version leaky_app_fixed.c should release allocated memory when it is no longer needed.

Typical workflow:

- Build both versions with debug symbols: gcc -g -O0 -Wall -Wextra -o leaky_app leaky_app.c and gcc -g -O0 -Wall -Wextra -o leaky_app_fixed leaky_app_fixed.c.
- Run the baseline: valgrind --leak-check=full --show-leak-kinds=all ./leaky_app
- Run the corrected version with the same options and compare the leak summary.

Expected interpretation: the baseline is used to demonstrate memory that remains allocated at program exit. In the fixed version, temporary buffers, record data, record objects, and the record-pointer array should all be released. The exact byte counts and error totals must be taken from the saved Valgrind output rather than estimated.

Evidence: logs/valgrind_report.txt.

6. System-call Analysis with strace

strace records system calls and signals observed while a process runs. It helps relate application behavior to operating-system activity, such as opening files, reading and writing data, memory mapping, and process termination.

Typical commands:

- Capture the full trace: strace -o logs/strace_full.txt ./buggy_app
- Summarize selected system calls: strace -c ./buggy_app

When interpreting the trace, focus on the system calls actually present in the log, their return values, and any failures. The summary can help identify which calls are frequent or consume a noticeable share of execution time; it should not be treated as proof of CPU hot spots by itself.

Evidence: logs/strace_full.txt and logs/strace_summary.txt.

7. Performance Profiling

The project includes a performance target program and saved performance summaries. The perf tool can collect execution statistics and, when supported by the kernel and permissions, sampling data. Histogram-style output can help compare where execution time or samples are concentrated.

Evidence available in the project: apps/perf_target.c, logs/perf_report.txt, logs/perf_stat.txt, and logs/hist_standard.txt. The raw file perf.data was not included in the submission copy because it was owned by root and not readable by the normal user; the text reports are retained. Exact numeric findings should be quoted from those reports.

8. Results Summary

Activity	Artifact	What it demonstrates
GDB	gdb_session.txt	Debugging workflow and execution observations
Valgrind	valgrind_report.txt	Memory errors/leaks reported by Memcheck
strace	strace_full.txt; strace_summary.txt	System calls and their summary
perf	perf_report.txt; perf_stat.txt; hist_standard.txt	Collected performance statistics and histogram output

The files listed above provide supporting evidence for the exercises. This report intentionally does not invent numerical results: the final values for leaked bytes, error counts, system-call counts, and performance metrics should be read from the corresponding logs.

9. Conclusion

This lab organizes a practical workflow for examining Linux programs from several perspectives: source-level debugging with GDB, heap-memory checking with Valgrind, system-call observation with strace, and performance review with perf reports. The comparison between the baseline and corrected memory-management examples is especially useful because it links a programming change to measurable tool output. Keeping source files and logs together makes the work easier to reproduce and review.

10. Submission Checklist

- Include source files and any required executable outputs in apps/.

- Include the saved text logs in logs/.
- Keep this PDF in bao_cao/.
- Before submission, verify that the report's statements match the actual log contents and add any required screenshots or exact numeric results specified by the instructor.

End of report